TOOLS.md — Fusion 360 MCP tool reference

Unit rule: every argument is in **millimeters**. Internally tools convert to Fusion's centimeters. When you (or Claude) write raw code, the API is in cm.

Face selection rule: `top/bottom` = max/min Z, `front/back` = min/max Y, `right/left` = max/min X. Chosen by extreme face-centroid position. Reliable on box-like parts; verify with `view_model` on complex geometry.

Core

`ping_fusion()`

Liveness + busy check, served on a **separate channel** from geometry execution — so it answers even while a long op runs on the main thread (a hung op no longer blocks ping). Reports `busy` and `current_job`.

`get_state()`

Cheap, guaranteed-fast doc state from the listener cache: active document name, lineage id, and body/sketch/component/timeline counts. Does **no geometry work** and is served without the main thread — safe to call during a heavy op or after a suspected hang. Use first for crash recovery. (For full per-body detail, `get_model_state`.)

`heartbeat()`

Crash-vs-busy check with explicit recovery instructions. Same liveness data as `ping_fusion`, plus guidance: if it errors, restart the add-in; if `busy=True`, poll `job_status`.

`run_async(code)`

Start a long operation **without waiting** — returns a `job_id` immediately. Poll `job_status(job_id)` for incremental progress and the final result. Use for anything expected to exceed ~90s (large patterns, heavy booleans, full-assembly rebuilds). Same injected helpers and unit rules. Call `progress(msg)` in your code to mark stages visible while it runs.

`job_status(job_id)`

Structured result of an async job: `status` (queued/running/ok/error/timeout), `elapsed_ms`, stdout collected **so far** (works mid-run), and traceback on failure. Poll until status is ok/error/timeout.

`run_fusion_code(code)`

Execute arbitrary Python in Fusion via the **bounded sync path**: waits up to ~100s and returns a structured envelope (status / elapsed / stdout / traceback). If the job outlives the wait it keeps running and stays pollable — the response tells you the job id. `adsk`, `app`, `ui` in scope, plus injected helpers (below). **Use `mm(x)` for every length** instead of hand-dividing by 10.

The docstring also documents the **sketch-plane axis mapping** (XY/XZ/YZ and offset planes) so the XZ→world-−Z inversion doesn't cost you a debugging cycle.

Injected helpers (available in `run_fusion_code` and `run_fusion_code_safe`, no import needed): - `mm(x)` / `cm(x)` — mm→cm for values you pass to the API. - `to_mm(x)` / `to_mm2(a)` / `to_mm3(v)` — cm→mm for reporting. - `assert_scale(max_mm=2000)` — raise if any body is 10×-too-big (units slip). - `total_volume()` — summed cm³ of all bodies; capture **before** a boolean. - `report_volume_delta(before, label)` — print the change and **scream if ~0** (silent no-op boolean). - `sliver_face_count(body)` — count sub-threshold faces (bad-boolean slivers). - `log_sketch_areas(sketch)` — print each profile area in mm² (catches scale slips). - `delete_face(face)` — delete one face the way Fusion accepts (`DeleteFaceFeatures`).

Recommended pattern for any cut/join in raw code:

```
_b = total_volume()
# ... do the boolean ...
report_volume_delta(_b, 'mycut'); assert_scale()
```

Known-unstable (refused by the listener): move-face via `defineAsFreeMove` (hard-crashes the host) and `defineAsOffset` (missing on this build). Use parametric edits / `offsetFeatures` / shell instead.

`run_fusion_code_safe(code)`

Transactional `run_fusion_code`: records the timeline marker and, on error, rolls back and deletes features created after it. Same injected helpers.

`clear_document()`

Delete all bodies, sketches, and timeline features. Returns design to empty.

Parameters

`set_parameter(name, value_mm)`

Create or update a user parameter (mm). Build parametric-first for anything that might change later.

`list_parameters()`

List user parameters and their expressions.

Primitives (root component, XY plane)

> **Auto-observability:** the material-removing tools (`drill_hole` , `hole_on_face` , `tapped_hole` , `shell_body`) automatically report the **volume delta** of the operation and **warn if it's ~zero** (a silent no-op cut that didn't touch the body). `shell_body` also reports sliver-face count. You no longer need to probe volume yourself to confirm a cut worked.

`create_box(length_mm, width_mm, height_mm, origin_x_mm=0, origin_y_mm=0)`

Rectangle on XY extruded up by height. Returns new body name.

`create_cylinder(diameter_mm, height_mm, origin_x_mm=0, origin_y_mm=0)`

Circle on XY extruded up.

`drill_hole(diameter_mm, x_mm, y_mm, depth_mm, body_index=0)`

Cut a hole straight down from XY at (x, y) to depth.

`fillet_edges(radius_mm, body_index=0, which="all")`

Round edges. `which` : `all` / `vertical` (vertical-running edges) / `top` (edges lying on the top face plane). Warns if nothing matched.

`shell_body(thickness_mm, body_index=0, open_top=True)`

Hollow to a wall thickness. `open_top=True` removes the top face (open box); `False` makes a closed hollow body.

`tapped_hole(thread_size, x_mm, y_mm, depth_mm, body_index=0, tap_drill_only=True)`

Tapped hole on the top (XY) plane, cutting down. `thread_size` e.g. `M3`, `M3x0.5`, `M4`, `M5`, `M6`, `1/4-20`, `10-32`, `#6-32`. - `tap_drill_only=True` (default): cuts only the correctly-sized tap-drill clearance hole — best for 3D-printed and hand-tapped parts. - `False`: also applies a **modeled** thread (visualization / CNC); best-effort, may be skipped with a warning if Fusion can't match the designation. Known tap-drill sizes include metric M1.6–M12 and common imperial; unknown sizes fall back to M3's drill — pass an explicit size you know is in the table, or ask to extend it.

Face-referenced features

`sketch_on_face(component_name="", which="top", body_index=0)`

Select a face by orientation and stash it. Empty `component_name` = root. Call `hole_on_face` immediately after (single-slot; don't interleave).

`hole_on_face(diameter_mm, u_mm, v_mm, depth_mm, through=False)`

Drill into the stashed face. `(u, v)` are mm offsets from the face centroid along the face's own axes. `through=True` cuts all the way.

Measurement & state

`get_model_state()`

Comprehensive design state in one call. Per body: name, validity, volume (cm³), bbox (mm), face/edge counts, and **sliver-face count** (flags a bad boolean). Plus body/sketch counts and the **timeline feature names**. Replaces hand-rolled introspection; makes crash recovery a one-call operation.

`measure_distance(which_a, which_b, body_index=0)`

True centroid-to-centroid distance (mm) between two oriented faces. Use to verify wall thickness, overall size, gaps — more trustworthy than the bbox.

`cross_section(body_index=0, axis="z", at_mm=None, span_axis="x", samples=41)`

Scan a line of points through a body and report solid/void as an **ASCII row** (`#`=solid, `.`=void) plus the **mm coordinates of every solid/void boundary**. The best non-visual diagnostic for trapped voids, missed cuts, and wall thickness. Holds the two non-span axes at their mid-value (or at `at_mm` along `axis`) and sweeps `span_axis` across the bbox. Example: `cross_section(axis="z", at_mm=10, span_axis="x")` → reads off wall positions along X at Z=10 mm.

`list_faces(body_index=0, max_faces=40)`

Faces with a stable **tempId**, centroid (mm), area (mm²), surface type, largest first. The tempId lets you re-select the same face later via `body.findByTempId(tempId)` in raw code — essential for multi-step face work.

`list_edges(body_index=0, max_edges=40)`

Edges with a stable **tempId**, length (mm), midpoint (mm), longest first. Use the tempId to target specific edges for fillets/chamfers in raw code.

Visual feedback

`view_model(orientation="iso")`

Orient (iso/top/front/back/right/left), fit, screenshot, and **return the image** so Claude sees it. Call after non-trivial operations to verify.

`set_view(orientation="iso")`

Reposition the camera without returning an image.

Components & occurrences

`create_component(name)`

New empty component as an occurrence at the origin. One per distinct part.

`list_components()`

All occurrences: fullPathName, component name, origin position (mm).

`box_in_component(component_name, length_mm, width_mm, height_mm, ox_mm=0, oy_mm=0, oz_mm=0)`

Box built inside a named component on its own XY plane.

`move_component(component_name, x_mm, y_mm, z_mm)`

Translate an occurrence's origin. Position BEFORE jointing.

`copy_component(component_name, count, dx_mm=0, dy_mm=0, dz_mm=0)`

Place `count` more occurrences of one definition in a linear pattern. Reuses the definition (edit once, all update) — ideal for repeated hardware.

`ground_component(component_name, grounded=True)`

Lock/unlock a component in space. Ground exactly one base per assembly, first.

Joints

Joints reference component **origins** (or explicit points). To pivot at a specific location, design the component so its local origin sits there, or use `joint_at_point`. Position parts first; joints override transforms after.

`rigid_joint(component_a, component_b)`

Lock two components rigidly at their current relative position.

`revolute_joint(component_a, component_b, axis="z")`

One rotational DOF about `axis` (x/y/z). Hinges, lids, arms.

`joint_at_point(component_a, ax_mm, ay_mm, az_mm, component_b, bx_mm, by_mm, bz_mm, motion="rigid", axis="z")`

Joint at explicit local points on each component. `motion`: rigid/revolute.

`list_joints()`

All joints and their motion type.

Export

`export_step(filepath)`

STEP of the whole design. Absolute path, `.step`.

`export_stl(filepath, body_index=0, quality="medium")`

STL of one body for 3D printing. `quality`: low/medium/high.

`export_dxf(filepath, sketch_index=0, component_name="")`

DXF of a sketch profile for laser/waterjet/flat stock. Selects sketch by index in root (or a named component).

Document persistence

Require being signed in with an **active project** in Fusion's Data Panel.

> **Document identity is now surfaced everywhere.** `get_state` and `get_model_state` both lead with the active document's **name + lineage id**, so you can see which document a `clear` / `save` / `open` will affect before acting. This prevents the worst error class — overwriting the wrong design.

`save_document(name="", create_new=False)`

Save the active doc and **always report which lineage was written**. - `create_new=True` → save-as a brand-new document/lineage named `name`. Use for a **new part** so it never lands in an unrelated design's history. - `create_new=False` (default) → unsaved doc saves-as `name`; already-saved doc saves a **new version of the current lineage**, printing the name + lineage id it versioned. **Warning:** this versions whatever lineage is open — if you opened part A but meant to save new part B, it buries B in A's history. When in doubt, `create_new=True` or check identity first.

`list_recent_designs(limit=15)`

Designs in the active project root, as `name :: id`.

`list_versions(design_id="")`

Read-only: enumerate the saved versions of a lineage (version number, id, date) so you can confirm what's recoverable after a bad save and point the user to the exact version to promote in the Data Panel. Empty `design_id` = active document's lineage.

`open_document(design_id)`

Open a design by id. **Changes the active document** — subsequent `clear` / `save` act on it. Reports the new lineage; confirm with `get_state()`.

Typical workflows

Single part: `clear_document` → `set_parameter`(s) → primitive(s) → features (`fillet_edges` , `shell_body` , `tapped_hole` , `sketch_on_face` + `hole_on_face`) → `view_model` → `get_model_state` / `measure_distance` → `export_stl` / `export_dxf` .

Assembly: `create_component` per part → `box_in_component` (or build geometry) → `ground_component` base → `move_component` others → `rigid_joint` / `revolute_joint` / `joint_at_point` → `view_model` → `save_document` .

Resume later: `list_recent_designs` → `open_document(id)` → modify → `save_document` (new version).

Known-unstable operations (refused by the listener)

The listener actively **refuses** these calls before they reach Fusion, because they have hard-crashed the host in practice (which kills the listener and requires a manual add-in restart). If you send code containing them, you'll get a `REFUSED (stability guard)` message instead of a crash:

Call	Why refused	Use instead
`*.defineAsFreeMove(...)` (move-face free move)	Hard-crashes the Fusion host	Edit the parametric feature, or rebuild the body
`*.defineAsOffset(...)`	Not reliably available on this build	`offsetFeatures` , or a shell / offset-plane extrude

If you find another operation that reliably crashes the host, add its call signature to `_DANGEROUS_PATTERNS` in `MCPListener.py` and document it here.

Diagnosing a hang or suspected crash

The listener runs jobs **asynchronously** and answers ping/state on a separate channel, so you can always tell what's happening:

1. `heartbeat()` (or `ping_fusion()`). If it responds with `busy=True` , a job is still running — `job_status(current_job)` for its partial output. If it errors/times out, the listener is down.
2. If a `run_fusion_code` call returned "STILL RUNNING" or a timeout envelope, the job kept going — poll `job_status(job_id)` rather than re-issuing it (re-issuing stacks another job behind the stuck one).
3. `get_state()` is always cheap and never blocks — use it to see the active document and counts even mid-operation.
4. If the listener is truly down: in Fusion, **Shift+S → Add-Ins → MCPListener → Run** to restart it, then `get_state()` to see what survived.

A job that exceeds its budget is flagged `timeout` in the envelope (the watchdog can't force-kill a hung main thread, but you get a clear signal instead of a dead channel).